



AURIX: Long distance communication

Department of Information Engineering
ICT for Industrial Applications

Group D:

Mahshid Shams
Department of Information Engineering
mahshid.shams@studenti.unipd.it

Laura Settimo
Department of Information Engineerig
laura.settimo.1@studenti.unipd.it

Piero Frassetto
Department of Information Engineering
piero.frassetto@studenti.unipd.it

Alessandro Lucatello
Department of Information Engineerig
alessandro.lucatello@studenti.unipd.it

Aim

Interface *AURIX TC375 Lite Kit* with *ARCELI HC-12* wireless module to enable long-distance communication. Enable wireless communication within *AURIX* board so as to exchange data.

1 Motivations and state of the art

Motivations

Efficient systems in areas like smart cities, environmental monitoring, and agriculture rely on long-distance communication that is necessary for real-time data capture and decision-making. It ensures safety of industrial settings by allowing remote operation of equipment as well as being crucial to disaster emergency teams.
The Internet of Things (IoT) ecosystem needs dependable long-distance communication for linking multiple devices, which is essential in applications such as smart grids, logistics, and healthcare. Dealing with these challenges helps to bridge the digital gap through provision of basic services and information in underserved areas thereby promoting worldwide interconnections.
This project is dependent on the HC-12 Wireless module which aids in long distance communication for exchanging temperature, humidity and proximity detection data between AURIX boards. It has a wide range of applications as it consists of one hundred channels that can be optimized to function under different conditions by adjusting the power at which they transmit. There is no limit to this chip's functionality because of its low power consumption hence it can operate over a long period especially when using battery-powered devices.
It also possesses a UART communication interface used for connecting with microcontrollers thus enabling rapid growth. Besides, the HC-12 provides an affordable way of achieving reliable long distance communication capabilities, thereby enhancing the overall functioning and success of the project.
On top of that, this wireless module functions within 433.4 MHz to 473.0 MHz band that we seleted 433.4Mhz because we used the first channel of 100 channels and covers up to 1 km area making it useful across multiple spheres due to its ability to send robust data over long distances.

State of the art

A wireless transceiver module referred to as HC-12 is used for long-range serial communication and can operate between 433.4 to 473.0 MHz. While several solutions have been developed, a few challenges still arise in using this popular device. By providing innovative approaches for these obstacles, varied applications from simple wireless communication projects to complex IoT systems will find the most suitable solution with the HC-12 module.

Leading Solutions	**Details**
Advanced Antenna Design	Use high-quality antennas such as parabolic or Yagi for directivity.
Frequency Hopping	The radios should change their transmission frequencies after a certain period.
Enhanced Modulation Techniques	Use suitable techniques such as FSK or QAM.
Custom Firmware	Developing custom firmware for having better performance.

Table 1: Leading solutions and their details

Challenges	Solutions
Range and Interference	- Use better antennas - Ensure direct visibility of destination points - Choose less crowded channels
Power Consumption	- Use power saving mode - Optimize transmission times - Use renewable energy sources
Data Integrity	- Use error checking techniques - Use components with good quality
Complexity of Integration	Using libraries and example codes available from the community can simplify the integration process (Raspberry, ..)

Table 2: Challenges and solutions

2 Proposed solution

In this section, we aim to describe the process of establishing communication between two *HC-12* antenna modules connected to the *Aurix* board. This report outlines the most crucial steps (indicated with a numbered list) we encountered and the reasons behind them.

How the Aurix board and HC-12 communicate?

The *HC-12* module contains MCU (Microcontroller Unit). Users do not need to program it in addition, which allows sending and receiving UART data. By simply transmitting UART data to the antenna module, we can establish wireless communication between two *HC-12* antenna modules [1].
As shown in the Figure 1, the *HC-12* module is used to replace the physical connection in half duplex communication. Modules can only work in half duplex state and cannot send and receive data at the same time.

1. In the list of preexisting projects in ADS, we looked for a project that could enhance UART communication. Under the guidance of *Infineon*'s engineers, we selected the `ASCLIN_UART_1 KIT_TC375_LK` project.

How to connect the pins?

For communication between the *Aurix* board and the *HC-12* module antenna, it is crucial to connect the TX pin of the *Aurix* board to the RX pin of the *HC-12* module, and vice versa.

2. The pins were connected between the *HC-12* module and the *Aurix* board as outlined in Table 3.

For TX and RX, we selected the pins `P33.12` and `P33.13` as indicated in the *Aurix* user manual.

3. We connected the SET pin to the GND pin to enable entry into AT command mode, which allows us to verify the correct connection of the antenna module to the *Aurix* board.

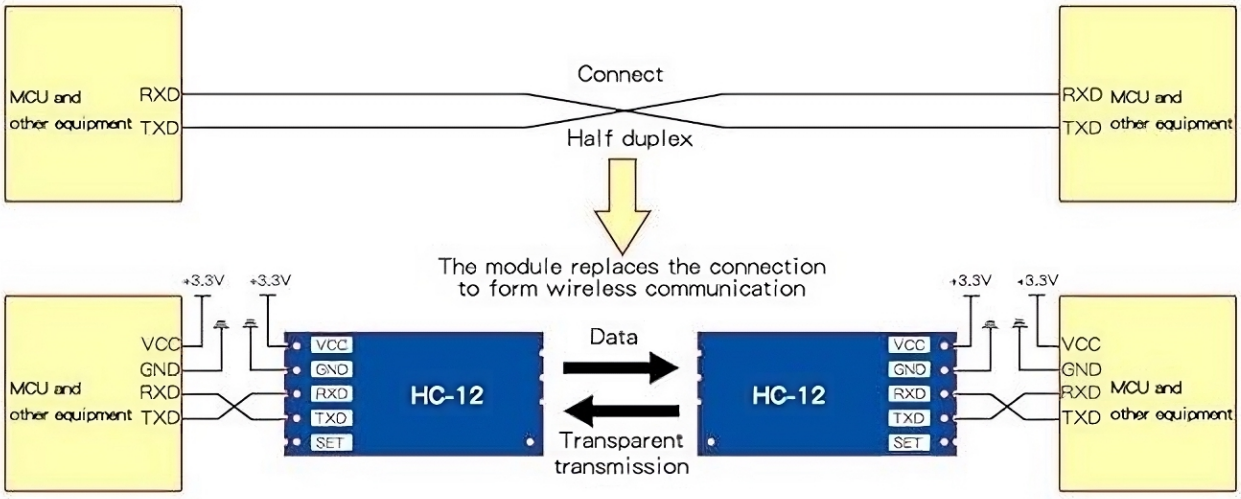


Figure 1: Replace the cable connection with the antenna module.

HC-12 antenna module	Aurix TLC375 LiteKit
+3V3	VCC
GND	GND
RXD	P33.12
TXD	P33.13
SET	GND

Table 3: Interfacing the pins between a *Aurix* and the *HC-12* antenna module

Changes to the code

All changes to the code are accurately tracked in the "README.txt" file. You can also review the changes in a GitHub repository. This report will explain the reasons behind these changes.
We began with the `ASCLIN_UART_1_KIT_TC375_LK` project, which enables internal UART communication in *Aurix* by sending the message "Hello World!" through pin `P15.15` and receiving it on the same pin.

4. Initially, we modified the definitions in the code found within "ASCLIN_UART.c" to address these aspects:

   - Set the **baud rate** to 9600 bps by changing the `UART_BAUDRATE` definition.
   - Adjust the RX and TX **pins** as previously discussed by changing `UART_PIN_RX` and `UART_PIN_TX` definitions.
   - Adjust the **size** of the receiving vector, which stores the incoming data, by changing the `SIZE` definition.
   - Adjust the expected **bytes** in reception by modifying the `g_rx` constant.

In the code, we set `g_rx` equal to 3 because, after sending the message "AT\r" to the antenna, we expect to receive "OK\r" in response. The pointer to `g_rx` is used in this function:

```
IfxAsclin_Asc_read(&g_ascHandle, g_rxData, &g_rx, TIME_INFINITE);
```

This function waits indefinitely to receive 3 bytes from the "Asclin_1" module on the *Aurix* board. After receiving them, it recorded them in the `g_rxData` file.

5. Before attempting communication between the two antenna modules, we verified that the UART communication between the two *Aurix* boards was working correctly as shown in Figure 1. After testing, the UART communication is confirmed to be working, ensuring the correct selection of pins.

6. The antenna module has been connected to the *Aurix* board. However, we have encountered some issues with the correct configuration of the antenna module, which will be explained and solved in Section 3.

This function sends the data stored in `g_txData`, and the size of the transmitted data is recorded in `g_count`:

```
IfxAsclin_Asc_write(&g_ascHandle, g_txData, &g_count, TIME_INFINITE);
```

This function allows us to send the AT command.

7. With the AT commands ”AT+RX” we checked that the two antenna modules had the same UART transmission mode, UART baud rate, communication channel, transmit power and other information [1].

8. Finally, we test the communication between the two antenna modules. We started transmission between the two antennas by disconnecting the SET pin from the GND.
We set one board as the transmitter, leaving only the `write` function, and the other as the receiver, leaving only the `read` function. We tested the system by sending a simple text message: ”Hello.” The successful reception of the message confirmed the functionality of the antenna communication.

3 Baudrate issue

Aim

Importing from *Aurix* Development Studio (ADS) the `ASCLIN_UART_1_KIT_TC375_LK` project. Modifying the project to correctly change the values to establish the connection with the wireless module (as we explain in the previous section). When we sending the ”AT” command through UART communication using the Aurix board, we should receive an ”OK” response from the antenna module [1]. This indicates that the antenna module is correctly connected to the *Aurix* board.

Problem

When using the debug mode in ADS, we received a response, but it is not the ”OK” message required to establish communication with the antenna.

Expression	Type	Value
> g_rxData	unsigned char *	0x70000008, "□8□"
> g_txData	unsigned char *	0x7000006c, "AT"
Add new expression		

Figure 2: ADS debug mode: the content we received is in the yellow cell.

After verifying the cable connection and the buffer size of the vector where the data is received, *Infineon*’s engineers suspected that the issue might be in the antenna’s baud rate, which differs from

the nominal rate.
To try to verify if the problem was in the antenna's baudrate, the *Infineon*'s engineers used a *Saleae* logic analyze to capture and display the logic signals from the `P33.12` and `P33.13` *Aurix* board's pins. This setup allowed us to monitor the signals received and transmitted by the antenna. From viewing the received logic signal, we discovered that:

- The baud rate of the antenna's Tx was correctly set at 9600 bps.
- The baud rate for the *Aurix*'s Tx differed from the baudrate specified in the code definition.

The issue was due to an incorrect setting of the baud rate within the baud rate function of the `ASCLIN_UART_1_KIT_TC375_LK` project.

How baud rate is set in Aurix board?

In Figure 3, it is shown that the baud rate is calculated using a dividers that divide an initial frequency f_A. The sequence of generated frequencies is calculated as follows:

$$f_{PD} = \frac{f_A}{(\text{BITCON.PRESCALER+1})}$$

$$f_{OVS} = f_{PD} \cdot \frac{\text{BRG.NUMERATOR}}{\text{BRG.DENOMINATOR}}$$

$$f_{SHIFT} = \frac{f_{OVS}}{(\text{BITCON.OVERSAMPLING+1})}$$

The following bit fields (registers) are available for the configuration of the 28-bit baud rate divider chain shown in Figure 3:

- `BITCON.PRESCALER` is the division ratio of the pre-divider.
- `BRG.NUMERATOR` is the numerator of the fractional divider.
- `BRG.DENOMINATOR` is the denominator of the fractional divider.
- `BITCON.OVERSAMPLING` is the division ratio of the baud rate post divider.

The overall formula is given as follows where f_{SHIFT} is equal to the baud rate:

$$\text{BAUDRATE} = \frac{f_A \times \text{NUMERATOR}}{(\text{PRESCALER} + 1) \times \text{DENOMINATOR} \times (\text{OVERSAMPLING} + 1)} \tag{1}$$

Changing the values of the previously listed registers can change the outcomes of equation (1).

Solution

Starting from equation (1) we tuned the values to obtain the correct value of the baud rate. In the *Aurix* user manual, we found an example (Figure 4) that shows how to configure the correct values to achieve the desired baud rate of 9600 bps but with a frequency $f_A = 100$ Mhz.

Assuming f_A = 100 MHz and (defined in line 8) prescaler = 10, oversampling = 16:
*fshift = (fclc/prescaler * 48/3125) / oversampling = 9600 Bd*

Figure 4: An example taken from the Aurix board user manual.

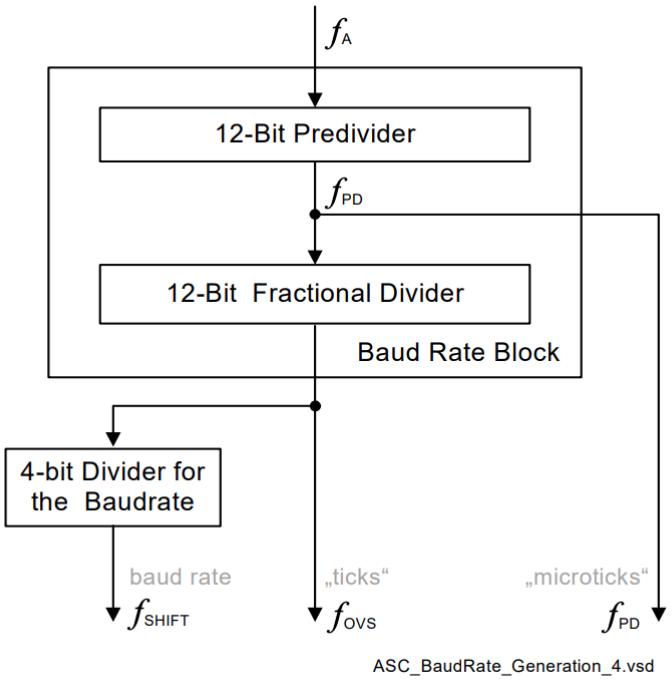


Figure 3: A diagram illustrating the calculation of baud rate taken from the *Aurix* board user manual.

However, we had to change the numerator's value (`BRG.NUMERATOR`) because, in the example, f_A is 100 MHz, whereas in our case, it is equal to 200 MHz. Therefore, we needed to divide the value of the numerator given in the example (48) by 2.

Working alongside *Infineon*'s engineers, we identified an error in a default project within ADS concerning the baud rate. We resolved the issue by consulting the user manual on how to calculate the baud rate and applying what we understood.

Conclusions

This experience allowed us to work with the Aurix board and collaborate with their engineers to address various challenges. Additionally, it enabled us to explore potential applications for such a communication system. For instance, a practical application might involve pressing a button on one board to trigger an LED on another board, which could be utilized in automation and remote control projects, facilitating interaction between different devices.

The *HC-12* antenna, with its 1 km range, opens up numerous possibilities for home automation, such as:

- Controlling lights
- Turning on electronic devices
- Remote vehicle control
- Transmitting temperature data

Furthermore, this system could be scaled to an industrial level to monitor and control machinery remotely.

References

[1] Hc-12 user manual. Seeed Studio. Accessed on July 5, 2024.